

Week 1: Complete 7-Day CI/CD Master Handbook

Your step-by-step roadmap from total beginner to production-grade GitHub Actions engineering.

Day 1: Foundations, Runners & Quality Gates

COMPLETED ✓

Core Concept: CI (Continuous Integration) automatically tests code on every push, while CD (Continuous Deployment) automatically delivers tested releases to servers.

Developer (git push) → GitHub Cloud Runner (Ubuntu VM) → Checkout → Install → Test (Exit 0=Pass / Exit 1=Fail)

Key Learnings:

- **The 5 Building Blocks:** Workflows (`.github/workflows/*.yaml`) → Triggers (`on:`) → Jobs (`jobs:`) → Runners (`runs-on:`) → Steps (`steps:`).
- **Marketplace Actions (`uses:`):** `actions/checkout@v4` downloads code onto the empty cloud runner. `actions/setup-node@v4` installs Node.js.
- **Exit Codes:** Terminal exit code `0` means success. Any exit code `1+` halts the workflow with a red ✖.
- **Golden Rule:** Never run persistent tools (e.g. `nodemon` or `npm start`) in CI. Commands must execute and exit.

Day 2: Context Objects, Interactive UI & Step Summaries

COMPLETED ✓

Core Concept: Dynamically adapt pipelines using GitHub's metadata and interactive UI controls.

Key Learnings:

- **Context Objects (`${{ github.* }}`):** Access dynamic data like `github.actor` (who ran it), `github.sha` (commit ID), and `github.ref_name` (branch).
- **Interactive Dropdowns (`workflow_dispatch.inputs`):** Creates self-service UI controls on GitHub where users can select environments (e.g., `development`, `staging`, `production`) or enter custom parameters.
- **Live Dashboards (`$GITHUB_STEP_SUMMARY`):** Appending markdown tables to this environment variable renders beautiful reports directly on the GitHub Actions summary page.
- **Secrets Masking:** GitHub automatically redacts repository secrets (e.g. `${{ secrets.MY_API_KEY }}`) with `***` in logs.

Day 3: Concurrency & Cancelling Stale Builds

DAY 3 (TOMORROW) ⌚

Core Concept: Save cloud bills and runner minutes by automatically cancelling outdated running pipelines when new commits arrive.

Push Commit 1 (Build Running 🟡) → Push Commit 2 → Commit 1 Automatically Cancelled 🔴 → Commit 2 Runs 🟢

Key Code Pattern:

```
# Cancels any ongoing build for the same branch when a new commit is pushed
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

💡 **Why Companies Demand This:** If a developer pushes 5 commits in 2 minutes, without concurrency GitHub runs 5 separate builds (wasting 20+ build minutes). Concurrency kills the old 4 runs and only builds the latest code!

Day 4: Status Check Functions & Self-Healing Pipelines

DAY 4 ⌚

Core Concept: Control step execution based on whether previous steps passed, failed, or were cancelled.

Status Function	When It Executes	Production Example
<code>if: success()</code>	Default. Runs only if all previous steps succeeded.	Proceed to deployment.
<code>if: failure()</code>	Runs ONLY if ANY previous step failed.	Send an alert to Slack/Discord or email the engineer.
<code>if: always()</code>	Runs ALWAYS, even if tests crash or are cancelled.	Upload crash logs and shut down temporary test databases.

Day 5: Code Quality Gates (ESLint & PR Merge Guards)

DAY 5 ⌚

Core Concept: Enforce high engineering standards automatically before code can be merged into the master codebase.

- **Static Code Analysis (ESLint):** Automatically checks JavaScript code for unused variables, missing semicolons, or security flaws.
- **Pull Request Protection:** Automatically runs tests when a PR is opened. If tests or linters fail, GitHub locks the "Merge" button.

Day 6: Multi-Job Pipelines & Dependency Graphs

DAY 6 ⌚

Core Concept: Divide complex systems into modular jobs (Lint → Test → Build → Deploy) connected with `needs:`.

```
[ Job 1: Lint & Code Style ] └─┬─> [ Job 3: Integration Tests ] ──> [ Job 4: Production Deploy ]
[ Job 2: Unit Tests          ] └─┘      (needs: [lint, test])      (if: branch = 'main')
```

Key Code Pattern:

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: [ ... ]

  test:
    runs-on: ubuntu-latest
    steps: [ ... ]

  deploy:
    runs-on: ubuntu-latest
    needs: [lint, test]      # Waits for BOTH lint and test to pass!
    if: github.ref == 'refs/heads/main'
    steps: [ ... ]
```

Day 7: Week 1 Production Capstone & 9 LPA Interview Q&A

DAY 7 ⌚

Core Concept: Build a complete end-to-end enterprise workflow from scratch and practice senior interview questions.

🔥 Top 9 LPA Interview Questions for Week 1:

1. Q: What is the difference between `uses` and `run`?

A: `uses` calls pre-packaged, reusable actions from the GitHub marketplace, while `run` executes arbitrary terminal/shell commands on the runner.

2. Q: How do you prevent wasted compute when rapid commits are pushed?

A: We define a `concurrency` group with `cancel-in-progress: true` tied to the branch ref.

3. Q: Why does GitHub Actions fail when a command exits with code 1?

A: Standard POSIX convention dictates that exit code 0 is success and any non-zero exit code represents a runtime error. GitHub monitors the process exit code to gate execution.

4. Q: How do you ensure cleanup steps run even if tests crash?

A: We attach the `if: always()` status check function to the cleanup step.